Drone flight control architecture

Recommended structure for commanding the PX4 drone from ROS 2 in the warehouse simulation

ASP Project · `ros2_ws/src/warehouse_bringup` · PX4 SITL + Gazebo Harmonic + ROS 2 Humble · October 2026

Summary. The current `takeoff_1m` node is a good proof of concept: it switches PX4 to Offboard, arms the drone and holds a fixed setpoint. To fly real paths through the warehouse, the same logic should be split into clear layers:

- an **offboard controller node**, the only node that talks to PX4, running a state machine and streaming setpoints;
- a **ROS 2 Action interface** (`GoTo` , `FollowPath`) to give it goals with feedback and cancellation;
- a **trajectory generator** that moves the setpoint smoothly instead of jumping to the target;
- a **mission node** on top that decides where to go, and later uses the depth camera.

1 Background: what Offboard mode is

PX4 always runs in exactly one **flight mode**, and the mode decides who provides the targets the controllers try to reach: a pilot (Manual, Position), PX4 itself (Takeoff, Land, Hold, Mission), or, in **Offboard** mode, an **external program**. In this project that program is a ROS 2 node. PX4 still does everything low-level: state estimation (EKF2), position, attitude and rate control, and the motor outputs. It just takes its targets from the node.

Handing control to an external program is risky, so PX4 enforces a few rules. They explain most of the structure of the code:

1. **Heartbeat.** `OffboardControlMode` must be published at more than 2 Hz. If it stops for longer than `COM_OF_LOSS_T` (default 1 s), PX4 triggers a failsafe (`COM_OBL_RC_ACT` : hold, land or return).
2. **Stream first, then switch.** PX4 refuses to enter Offboard unless setpoints are already arriving.
3. **Valid estimate.** Position control needs a valid local position (`xy_valid` , `z_valid`).
4. **Arming is separate.** Entering Offboard does not start the motors. The vehicle must also be armed.

Positions are in PX4's local **NED** frame (North, East, Down), so **up is negative z**: "1 m above the start" means `z = z_start − 1.0` .

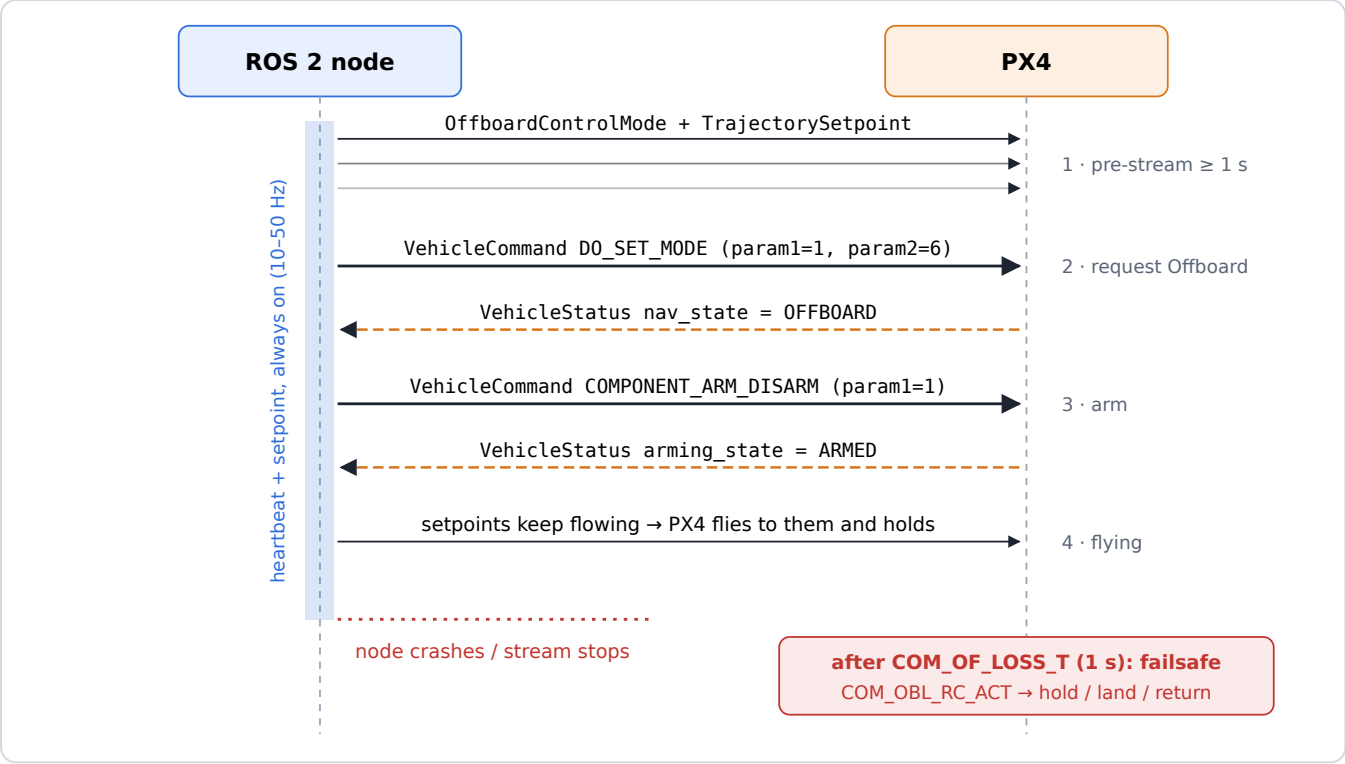


Figure 1. The Offboard handshake. This is the sequence the current `takeoff_1m` node implements. The heartbeat must never stop while PX4 is in Offboard.

2 The current node and its limits

`warehouse_bringup/takeoff_1m.py` does three different jobs inside one 10 Hz timer callback:

Job	Where	Status
Talking to PX4 (heartbeat, mode, arming)	`_control_loop`, `_publish_vehicle_command`	Works. Retries every 1 s.
Deciding the target	`_publish_setpoints`	One hard-coded point. No path, no speed control.
Mission logic	none	No landing, no next goal, no end state.

Things to fix whatever structure you choose.

- **The target is 10 m, not 1 m.** Line 98 uses `start_z - 10.0`, while the node name, log message and report say 1 m. Inside a warehouse 10 m may reach the ceiling.
- **No controlled shutdown.** On Ctrl-C the stream just stops and PX4 falls into the failsafe. It's better to command a landing first.
- **Hard-coded values.** Altitude, rate and speed should be ROS parameters, so they can be set from the launch file.
- **Simulated time.** `/clock` is bridged, but no node sets `use_sim_time`.

3 Recommended architecture

The core idea is **one owner per responsibility**. Exactly one node publishes on `/fmu/in/*`, so two programs can never fight over the drone. Everything that decides *where* to go talks to that node through a well-defined interface, never to PX4 directly.

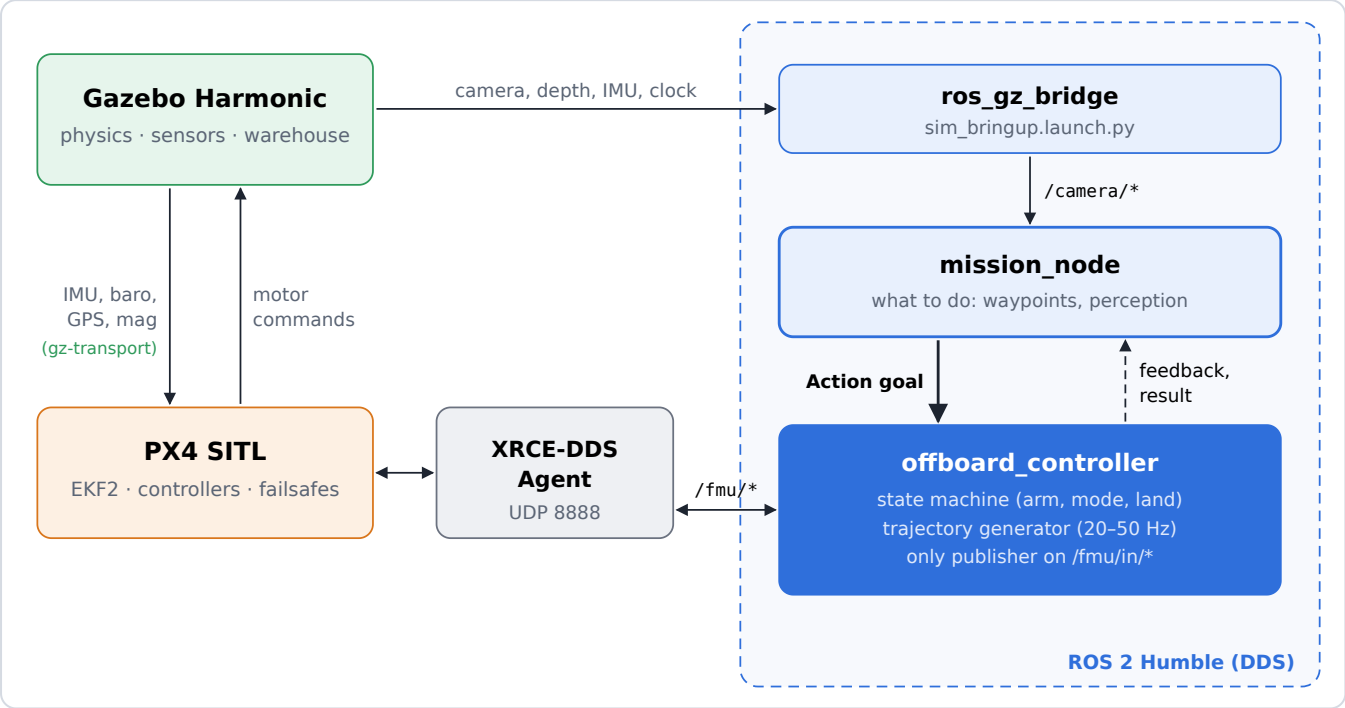


Figure 2. The whole system. The drone state comes from PX4 through the XRCE-DDS Agent (`/fmu/out/*`). Raw sensor data, mainly the cameras, comes from Gazebo through `ros_gz_bridge`. Only `offboard_controller` sends commands to PX4.

Four layers, four questions

Another way to read the architecture: each layer answers one question and runs at its own speed. Each layer only needs to know about the layer right below it.

LAYER	ANSWERS	OUTPUT	RATE
Mission mission_node	What should the drone do next?	goals: “GoTo shelf B2”, “inspect aisle 3”	on events
Trajectory offboard_controller	How do we get there, how fast?	moving setpoint: position + velocity	20–50 Hz
PX4 interface offboard_controller	Is PX4 ready and listening?	heartbeat, mode switch, arm / land commands	≥ 2 Hz
Flight control PX4 (unchanged)	How do I fly to this setpoint?	attitude, thrust, motor outputs	hundreds of Hz

Figure 3. Layers of responsibility. Layers 2 and 3 live in the same node. Keep them in separate classes or modules so that the trajectory maths can be unit-tested without ROS.

4 The offboard controller node

This node replaces `takeoff_1m`. Instead of a chain of `if` statements in the timer callback, it runs an explicit **state machine**. On every tick (20–50 Hz) it does two things, always in this order:

1. **Publish the heartbeat and the current setpoint**, whatever the state. This keeps PX4 happy.
2. **Advance the state machine**: check `VehicleStatus` and `VehicleLocalPosition`, send commands, change state.

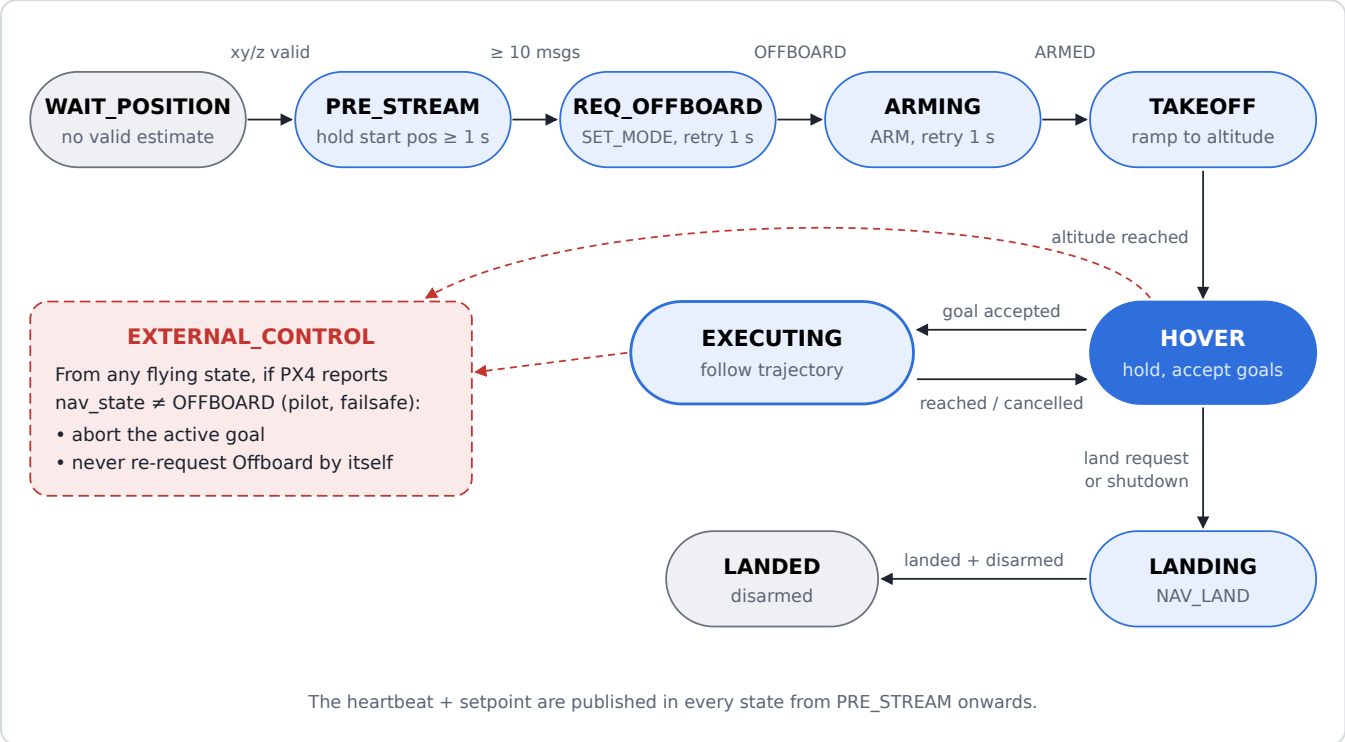


Figure 4. Controller state machine. The first four states are what `takeoff_1m` already does implicitly. The new parts are TAKEOFF as its own phase, HOVER as the resting state that accepts goals, a proper LANDING, and the reaction to PX4 leaving Offboard.

> **Important safety rule.** If the drone leaves Offboard while flying (a pilot switch or a PX4 failsafe), the node must **not** automatically switch it back. Someone or something took control on purpose. The current node re-requests Offboard every second whenever `_offboard_active` is false, which is fine only before takeoff.

Sketch of the control loop

```
class State(Enum):
    WAIT_POSITION = auto(); PRE_STREAM = auto(); REQ_OFFBOARD = auto(); ARMING = auto()
    TAKEOFF = auto(); HOVER = auto(); EXECUTING = auto(); LANDING = auto()
    LANDED = auto(); EXTERNAL_CONTROL = auto()

def _tick(self):                        # timer, e.g. 1 / self.rate_hz
    if self._state is State.WAIT_POSITION:
        return                          # nothing valid to stream yet
    self._publish_heartbeat_and_setpoint(self._traj.current())   # ALWAYS first
    self._traj.step(self._dt)           # move the setpoint along the path
    handler = self._handlers[self._state]   # one small method per state
    new_state = handler()
    if new_state is not self._state:
        self.get_logger().info(f'{self._state.name} -> {new_state.name}')
        self._state = new_state
```

Parameters and shutdown

- Declare tunables as ROS parameters: `rate_hz` (default 20), `takeoff_altitude` (1.0 m), `cruise_speed` (0.5 m/s), `acceptance_radius` (0.15 m). They can then be set from `sim_bringup.launch.py`.
- Pass `use_sim_time:=true` in simulation so timing follows Gazebo's `/clock`.
- On shutdown, if airborne, send `VEHICLE_CMD_NAV_LAND` and keep spinning until landed, or at least send the command once before exiting. Don't rely only on the offboard-loss failsafe.

---

5 Commanding the controller: ROS 2 Actions

ROS 2 offers three ways for nodes to communicate. A flight segment takes seconds, can fail half-way and may need to be stopped. That is exactly what an **Action** is designed for.

Mechanism	Behaviour	Fit for “fly to X”
Topic	Fire-and-forget stream	No confirmation, no progress, no way to know when it's done.
Service	Request → single reply	Meant for quick calls. Blocking for 20 s of flight is wrong.
Action	Goal → accepted/rejected → feedback stream → result. Can be cancelled.	**Designed for long-running tasks like this one.**

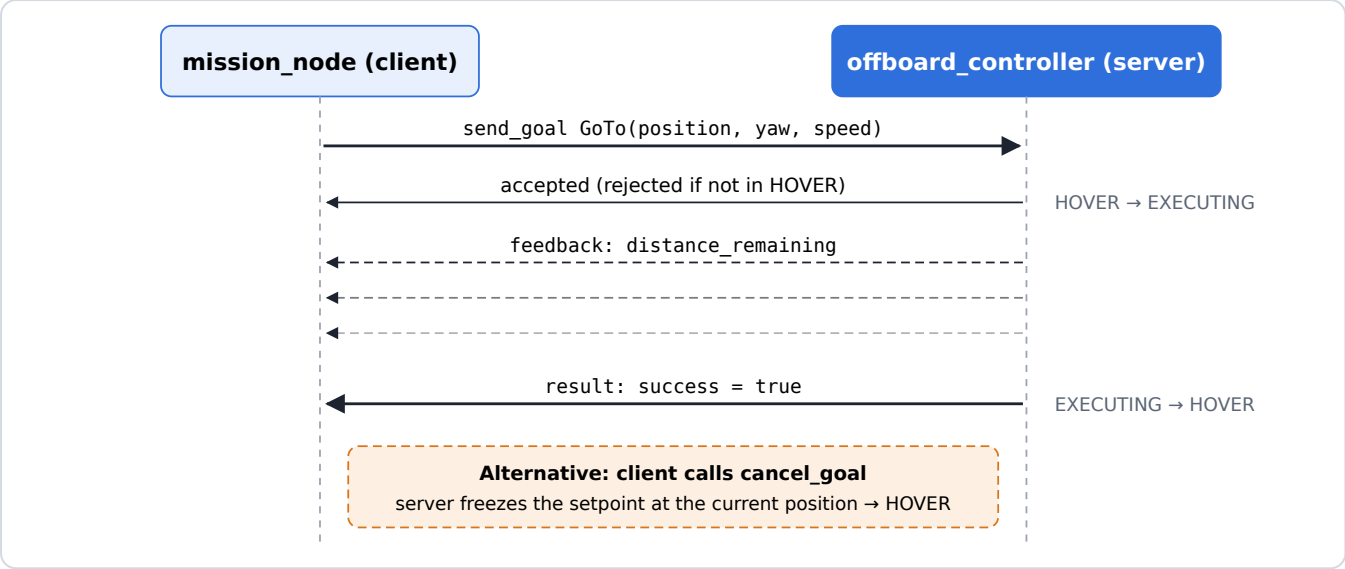


Figure 5. Life of one GoTo goal. The mission node can wait for the result, watch the progress, or cancel, for example when the depth camera sees an obstacle.

Interface definitions

Actions and messages must be defined in a separate **interface package** built with `ament_cmake`, because a pure Python package can't generate them. For example, `warehouse_interfaces/action/GoTo.action`:

```
# Goal: target in PX4 local NED frame, metres (z negative = up)
float32[3] position
float32    yaw          # rad, NaN = keep current heading
float32    speed        # m/s, 0 = use cruise_speed parameter
---
# Result
bool   success
string message
---
# Feedback
float32 distance_remaining
```

A `FollowPath.action` can take a `nav_msgs/Path` (a list of poses) as its goal and report the index of the current waypoint as feedback.

---

6 Trajectory generation

Today the node sends the **final target** straight away. PX4 then flies there as fast as its limits allow (`MPC_Z_VEL_MAX_UP`, `MPC_XY_VEL_MAX`, …), so you decide *where* the drone ends up, but not *how* it gets there. Among shelves, the path between points matters as much as the end point.

The fix is a **moving setpoint** (a “carrot”). On every tick the setpoint moves a small step along the path towards the goal, at the requested speed. The drone follows the carrot closely, so its speed and path are under your control.

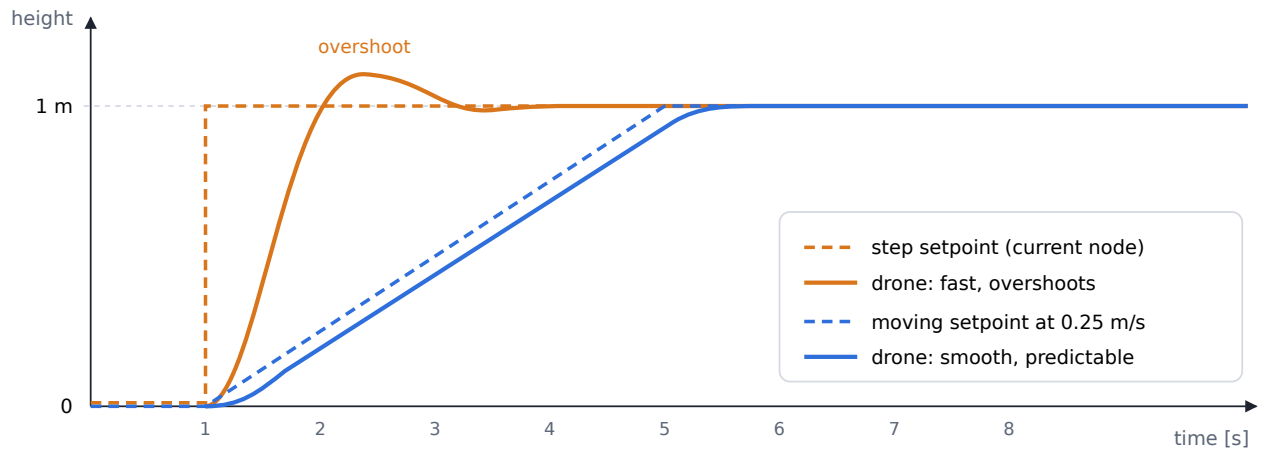


Figure 6. Step vs moving setpoint (illustrative). With the step, PX4 accelerates hard and overshoots the target. With the moving setpoint, the speed is set by the node and the drone tracks the reference closely. The same idea applies horizontally along the aisles.

Sending the carrot's **velocity** together with its position (feed-forward) reduces the tracking lag further. Keep this maths in a plain Python module with no ROS imports, so it can be tested with `pytest`:

```
# trajectory.py: no ROS imports
import numpy as np

class LinearTrajectory:
    """Moves a setpoint towards a goal at constant speed."""
    def __init__(self, start, speed):
        self.sp = np.asarray(start, dtype=float)
        self.goal = self.sp.copy()
        self.speed = speed
        self.vel = np.zeros(3)

    def set_goal(self, goal, speed=None):
        self.goal = np.asarray(goal, dtype=float)
        if speed: self.speed = speed

    def step(self, dt):
        delta = self.goal - self.sp
        dist = np.linalg.norm(delta)
        if dist <= self.speed * dt:        # arrived: snap and stop
            self.sp, self.vel = self.goal.copy(), np.zeros(3)
        else:
            self.vel = delta / dist * self.speed
            self.sp = self.sp + self.vel * dt
        return self.sp, self.vel
```

In the controller, put `sp` into `TrajectorySetpoint.position` and `vel` into `.velocity`, and set `OffboardControlMode.velocity = True` alongside `position = True`. TAKEOFF then becomes a goal straight above the start point. A goal counts as reached when the *drone's measured position* (not the setpoint) is within `acceptance_radius`.

> Later the linear segment can be replaced by something smoother (trapezoidal velocity profile, splines through the waypoints) without touching the state machine or the Action interface. This is the payoff of keeping the layers separate.

7 The mission node

The mission node holds the high-level plan and nothing else: it never publishes to PX4. A first version can be a hard-coded list of waypoints flown with `GoTo` goals one after another, followed by a land request. Later it can:

- load waypoints from a YAML file that describes the warehouse aisles;
- subscribe to `/camera/depth/points` and cancel or replan a goal when an obstacle is too close;
- trigger actions at waypoints, such as taking an image of a shelf.

8 Alternatives considered

Approach	Pros	Cons
Offboard from ROS 2 (recommended)	Full control, reacts to perception, Python is fine at 20–50 Hz, same code for simulation and the real drone.	You must handle the safety logic yourself (the state machine above).
PX4 Mission mode	PX4 flies uploaded waypoints by itself. Very robust.	Fixed routes, can't react to the camera, made for GPS missions (awkward indoors).
PX4 ROS 2 Interface Library (`px4_ros2_cpp`)	Your code registers as a real PX4 flight mode, and PX4 handles failsafes and mode switching.	C++ only, more setup. A good next step if the project grows.
Everything in one node (current)	Simple to start with.	Hard to extend and test, and mission logic gets mixed with the safety logic.

9 Suggested layout and migration steps

```
ros2_ws/src/
├── warehouse_interfaces/          # ament_cmake: action/GoTo.action,
action/FollowPath.action
├── warehouse_bringup/             # launch files, rviz configs (as today)
│   └── launch/sim_bringup.launch.py   # + starts offboard_controller / mission_node
└── warehouse_flight/              # ament_python
    └── warehouse_flight/
        ├── offboard_controller.py # state machine + PX4 I/O + GoTo action server
        ├── trajectory.py          # pure maths, unit-tested
        └── mission_node.py        # waypoint list → GoTo goals
```

1. **Fix the current node**: 1 m target, parameters, `use_sim_time`, land on shutdown.

2. **Refactor into the state machine** with explicit TAKEOFF, HOVER and LANDING states, still with a fixed target. Check that behaviour hasn't changed.
3. **Add** `trajectory.py` and make TAKEOFF use the moving setpoint. Add unit tests.
4. **Create** `warehouse_interfaces` and the `GoTo` action server. Test it with `ros2 action send_goal` from the terminal.
5. **Write** `mission_node`, which flies a list of waypoints through the aisles and lands.
6. **Add perception**: cancel or replan goals using the depth point cloud.